

Project Synopsis
on
HireLens: AI-Powered Resume Screening, Skill Gap Analysis
& Job Matching System

Submitted in partial fulfillment
for the award of the degree of
Bachelor of Technology
in
Computer Science Engineering
(Artificial Intelligence & Machine Learning)

Submitted By

Aryan	28240533
Nitish	28240529

Under the Supervision of
Prof. (Dr.) Devendra Parsad
HOD CSE AI & ML

Panipat Institute of Engineering & Technology,
Samalkha, Panipat

Affiliated to
Kurukshetra University Kurukshetra, India
(2025-2026)

Supervisor's Consent

The synopsis of final year project work titled HireLens: AI-Powered Resume Screening, Skill Gap Analysis & Job Matching System by the students' group id..... has been written with my consent and every section of this synopsis report is reflecting the work to be carried out by the group.	<i>(Signature of supervisor with date)</i>
---	---

Department Project Evaluation Committee (DPEC) Remarks

The project is by DPEC. The group is advised to submit progress of the project work in progress presentation1 to be held on.....

OR

The project is by DPEC. The group is advised to submit the synopsis report again after making changes as suggested by DPEC on

Name & Signature of DPEC member (s) with date

HireLens: AI-Powered Resume Screening, Skill Gap Analysis & Job Matching System

1. Introduction

Corporate recruitment pipelines routinely process hundreds of candidate resumes for every open position. Manual resume screening is time-consuming, labor-intensive, and inherently subjective. Clinical and industrial genomics pipelines similarly encounter complex data variants that impede decision making; in talent acquisition, legacy computational predictors—such as traditional Applicant Tracking Systems (ATS)—rely strictly on exact keyword matching, frequency counting, or rigid string lookup. Consequently, these traditional predictors rely on surface-level keyword features, limiting their applicability to exact phrasing and reducing generalizability across varied candidate terminology.

Recent advances in foundation language models have introduced Sentence-Transformers (SBERT), a largescale NLP embedding framework. SBERT enables **zeroshot variant effect prediction** by computing the difference in vector space representations between candidate experience and job requirements—a method grounded in deep semantic information theory rather than superficial keyword matching. This project implements an end-to-end system that integrates SBERT into a web accessible platform for real-time suitability scoring, leveraging structured skill ontologies and SQLite audit trails to provide interpretable, evidence-based outputs. The work bridges cutting-edge artificial intelligence with recruitment technology to support precision hiring through scalable, reproducible inference.

2. Objective

The primary objective is to **deploy the Sentence-BERT genomic/text foundation model** for candidate resume suitability scoring using a zeroshot dense vector similarity strategy. A secondary objective is to develop a **full stack web application** that enables users to input resume text, retrieve structured skill categories, fetch non-capturing regex contact audits, and compare candidate skills against job requirements—thereby facilitating transparent, clinically aligned interpretation and access to better reliable results without compromising the quality and accuracy in context to modern recruitment data values. It would also allow identify skill gaps based on domain variation and category handling, studying the effect and match suitability better.

3. Scope

The scope of this project is as below:

- **Theoretical Foundation:** Study of language models, vector effect prediction, and the algorithmic basis of semantic matching.

- **Model Integration:** Deployment of the pretrained SBERT model via a high-performance vector backend.
- **Data Pipeline:** Integration with document parsing APIs (pdfplumber for reference PDF layouts, python-docx for DOCX data).
- **Inference System:** Implementation of a FastAPI endpoint that accepts candidate specifications (resume text, job description, custom weights) and returns suitability scores.
- **Frontend Application:** Development of a Next.js / Streamlit interface for role search, Plotly skill visualization, and side by side comparison of AI vs. traditional keyword labels.
- **Evaluation Framework:** Benchmarking HireLens predictions against high confidence test sets to assess accuracy, calibration, and failure modes.
- **Reproducibility & Experiment Tracking:** Implement deterministic evaluation protocol including fixed random seeds, consistent data shuffling, and benchmark settings.

4. Architecture

- **Backend Stack:** Python, FastAPI, SQLite (ats_history.db), Transformers & Sentence-Transformers library
- **Frontend Stack:** Streamlit, Plotly Radar Charts, Pandas, Tailwind CSS / Shadcn UI
- **Genomic & Text Data Sources:** pdfplumber (for layout-aware PDF sequences), python-docx (for structured DOCX paragraphs)
- **Prediction Logic:**
 - Extract text window centred on candidate content
 - Compute dense 384D vector log embeddings using SBERT (all-MiniLM-L6-v2)
 - Output Cosine Similarity & Hybrid score; classify as Strong Match if score > empirically tuned threshold.

5. Methodology

This project follows an end to end MLOps pipeline—from model deployment to user facing inference.

5.1 Data Preparation and Integration

- **Input Specification:** User provides candidate resume file (PDF/DOCX), target job role, reference text, and alternate parameters.
- **Reference Sequence Fetching:** Query pdfplumber parser API to retrieve text window centred on document.
- **Contact Audit Lookup:** Use non-capturing regex `(?:\+91[\-\\s])?[6-9]\d{9}` to fetch clean email and 10-13 digit mobile numbers.

5.2 Model Inference

- **Tokenization:** Convert text string to token IDs using SBERT's native tokenizer.
- **Forward Pass:** Run forward embedding passes (candidate resume and job description) via PyTorch.
- **Scoring:** Compute Cosine Similarity = $(A \cdot B) / (\|A\| \times \|B\|)$.
- **Classification:** Apply hybrid threshold (ATS Score = 0.45 SBERT + 0.35 Skill + 0.20 TF-IDF) derived from benchmark analysis for binary decision.

5.3 Deployment & Monitoring

- **Serverless Backend:** Deployed on FastAPI / Streamlit Cloud preloaded with SBERT weights.
- **ColdStart Mitigation:** Use persistent background process keeping server warm during active usage.
- **API Contract:** RESTful POST endpoint returning JSON with prediction, score, skill gap match, and metadata.

5.4 Frontend Visualization

- Role search → Skill category breakdown → Resume input → Realtime prediction
- Display: Extracted text, ClinVar/Skill labels, SBERT score, confidence radar indicator
- Responsive UI with error handling for invalid files or format mismatches

6. Conclusion

This project demonstrates a production grade integration of a state-of-the-art foundation model into a clinically relevant decision support tool. By leveraging SBERT's zeroshot capabilities, the system bypasses the need for finetuning while delivering interpretable predictions grounded in computational linguistics. The modular architecture—separating inference, data retrieval, and user interface—ensures scalability, maintainability, and extensibility.

6.1 Key Highlights

- **Cutting Edge AI:** First known undergraduate implementation of SBERT dual-vector matching for ATS variant interpretation.
- **Full Stack Integration:** Seamless fusion of parsing APIs, Transformer inference, and modern web frameworks.
- **Clinical Alignment:** Direct comparison with skill taxonomies enables immediate validation and trust calibration.
- **Reproducibility:** Entire pipeline containerized; code open for audit and extension.

6.2 Limitations

- **Assembly Dependency:** Requires standard English text; multi-language inputs need translation pre-processing.

- **Context Window:** Fixed token window may miss distal regulatory details in extremely lengthy document attachments.
- **ColdStart Latency:** Initial model loading can take 2–5 seconds due to memory allocation.
- **Binary Output:** Current default threshold focuses on binary classification (Match vs Non-Match).

7. Future Scope

- **Uncertainty Quantification:** Integrate Bayesian or conformal prediction for calibrated confidence intervals.
- **OCR Integration:** Benchmark performance on scanned image PDFs using Tesseract OCR.
- **MultiVariant Analysis:** Extend to multi-resume candidate batch ranking and vector database (FAISS/Qdrant) search.
- **Production Optimization:** Replace HuggingFace backend with TensorRT-LLM for faster inference.
- **Interpretability Module:** Visualize attention maps or insilico mutagenesis profiles over input sequence.

Hardware/Software requirement

- **Programming Language:** Python, TypeScript
- **ML Framework:** PyTorch, HuggingFace Transformers, Sentence-Transformers
- **Cloud Platform:** Modal / Streamlit Cloud
- **Frontend:** Streamlit, Next.js, React, Tailwind CSS
- **Backend API:** FastAPI
- **Genomic / Parsing Libraries:** pdfplumber, python-docx, Biopython, requests
- **Monitoring:** Logging via SQLite; optional Weights & Biases for experiment tracking
- **Development Environment:** VS Code / PyCharm
- **Hardware:** Standard x86_64 CPU / Serverless GPU via Modal

References

- [1] Reimers, N., et al. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. arXiv preprint arXiv:1908.10084.
- [2] Landrum, G. A., et al. (2018). ClinVar: Improving access to variant interpretations and supporting evidence. Nucleic Acids Research, 46(D1), D1062–D1067.
- [3] Karolchik, D., et al. (2014). The UCSC Genome Browser database: 2014 update. Nucleic Acids Research, 42(D1), D764–D770.

- [4] Brown, T. B., et al. (2020). Language Models are FewShot Learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
- [5] He, K., et al. (2016). Deep Residual Learning for Image Recognition. *Proceedings of the IEEE CVPR*, 770–778.
- [6] Kircher, M., et al. (2014). A general framework for estimating relative pathogenicity of genetic variants. *Nature Genetics*, 46(3), 310–315.
- [7] Cheng, J., et al. (2023). Scaling Laws for DNA Language Models. *International Conference on Learning Representations*.
- [8] Devlin, J., et al. (2019). BERT: Pre-training of Deep Bidirectional Transformers. *NAACL-HLT*.
- [9] Streamlit Documentation (2024). Building Interactive Machine Learning Apps. <https://docs.streamlit.io/>
- [10] FastAPI Framework (2024). High Performance Python Web APIs. <https://fastapi.tiangolo.com/>